

Full Name: _____

CSO Practice Problems (Spring 2026)

Instructions:

- The actual final takes 100 minutes. This mock practice has more questions and should take you 120 minutes.
- This exam is **closed book**, except that you may bring a single page of prepared note.

1 (xx/16)	2 (xx/10)	3 (xx/20)	4 (xx/6)	5 (xx/12)	6 (xx/10)	7 (xx/12)	8 (xx/8)	9 (xx/6)	Total (xxx/100)

This exam assumes 64-bit x86 hardware (little Endian) unless otherwise mentioned.

1 Machine Representation (16 points, 2 points each):

Multiple choices. Circle *all* answers that apply. See Appendix 10 for a quick reference of x86 ISA.

A. Suppose register `%rbx` corresponds to some C variable `v`. Given x86 instruction `addq $4, %rbx`, what are *all* the potential types of `v` and the corresponding C statement of the instruction?

1. type: `int` , C statement: `v +=4;`
2. type: `unsigned int` , C statement: `v +=4;`
3. type: `long` , C statement: `v +=4;`
4. type: `unsigned long`, C statement: `v +=4;`
5. type: `int *` , C statement: `v +=4;`
6. type: `int *` , C statement: `v++;`
7. type: `long *` , C statement: `v +=4;`
8. type: `long *` , C statement: `v ++;`
9. none of the above

B. Suppose registers `%rdi` and `%rsi` store to C variables `x` and `y`, respectively. Given x86 instruction `movq (%rdi, %rsi, 8), %rax`, what are the most likely types of `x` and `y`, respectively?

1. `long` and `long`
2. `long *` and `long`
3. `long *` and `long *`
4. `int` and `int`
5. `int *` and `int`
6. `int *` and `int *`
7. none of the above

C. Which of the following x86 instructions change the value of register `%rsp`?

1. `ret`
2. `pushq %rax`
3. `popq %rbx`
4. `call`
5. `movq %rax, (%rsp)`
6. none of the above

D. Which of the following x86 instructions read from or write to memory?

1. `ret`
2. `pushq %rax`
3. `popq %rbx`
4. `call`
5. `addq %rax, %rbx`
6. none of the above

E. Suppose register `%rsi` stores C variable `y` of some pointer type. Which of the following instructions dereference the pointer `y`?

1. `leaq (%rsi), %rax`
2. `movq (%rsi), %rax`
3. `movq %rsi, %rax`
4. `subq %rax, (%rsi)`
5. `subq %rax, %rsi`
6. none of the above

F. Consider the following recursive function:

```
void foo(int c) {  
    if (c <= 0)  
        return;  
    foo(c-1);  
}
```

What is the minimum stack size (in bytes) needed in order to execute `foo(128)` successfully¹?

1. 8096
2. 1024
3. 512
4. 128
5. any value

G. Which of the following statements on segmentation faults are true?

1. Performing any out-of-bounds array access will result in an immediate segmentation fault.
2. Segmentation faults only occur when an instruction tries to write to memory.
3. Dereferencing a null pointer will always result in a segmentation fault.
4. Performing pointer arithmetic will always result in a segmentation fault.
5. none of the above

H. Suppose *local variable* `a` is defined as `int a[16];` Which of the following statements are true?

1. `a` takes up space on the stack.
2. `a` takes up space on the heap.
3. `subq $64, %rsp` allocates space for `a` and `addq $64, %rsp` de-allocates space for `a`.
4. `addq $64, %rsp` allocates space for `a` and `subq $64, %rsp` de-allocates space for `a`.
5. none of the above

¹You should assume that the compiler does not perform tail call optimization to avoid allocating a stack frame per function call.

2 Basic C (10 points, 2 points each)

Answer the following multiple-choice questions. Circle *all* answers that apply.

A. In the following code, what's the most likely outcome of line 4?

```
1: int a = 5;
2: int *p;
3: p = (int *)a;
4: printf("%d\n", *p);
```

1. It will produce a compilation error.
2. It will cause a segmentation fault.
3. It will print 5.
4. It will print some memory address.
5. None of the above

B. In the following code, what should be at line 3 in order for character 'd' to be printed at line 4?

```
1: char *s = "abcdef";
2: char v;
3: v = _____;
4: printf("%c\n", v);
```

1. `v = s+3;`
2. `v = *(s+3);`
3. `v = s[3];`
4. `v = *(++s);`
5. `v = ++s;`
6. None of the above

C. What is the output of the following code snippet?

```
char a[5] = {0, 0, -1, -1, -1};  
int *p;  
p = (int *) (a + 1);  
printf("%d\n", *p);
```

1. a positive number
2. a negative number
3. 0
4. Segmentation fault
5. Compilation error

D. Suppose `x` is of type `unsigned int`. Which of the following statement computes 0 if and only if the i -th bit of `x` (starting from the left) is zero? (The 0-th bit corresponds to the most significant bit).

1. `x & 0x80000000`
2. `(x << i) >> i`
3. `x & (1 << (31-i))`
4. `x | (0x80000000 >> i)`
5. `x & (0x7fffffff >> i)`
6. None of the above

E. Suppose `x` is of type `unsigned int`. Which of the following statement sets the i -th bit of `x` (starting from the left) to be one? (The 0-th bit corresponds to the most significant bit).

1. `x &= (1 << i)`
2. `x |= (1 << i)`
3. `x |= (1 << (31-i))`
4. `x &= (1 << (31-i))`
5. `x &= ~(1 << (31-i))`
6. None of the above

3 C to assembly (20 points):

Ben Bitdiddle wrote the following `swap` function to swap two long integers. Its corresponding assembly code is also shown below.

```
1 void
2 swap(long *a, long *b)
3 {
4     long tmp;
5     tmp = *a;
6     *a = *b;
7     *b = tmp;
8 }
```

```
1 swap:
2 movq (%rdi), %rax
3 movq (%rsi), %rdx
4 movq %rdx, (%rdi)
5 movq %rax, (%rsi)
```

A. (2 points) Where are function arguments `a` and `b` stored at, respectively?

B. (4 points) For each C statement, which is its corresponding assembly instruction (or set of assembly instructions)? Fill in the line number of the assembly instructions that correspond to each C statement in the table below.

C line number	Assembly line number
5	
6	
7	

Ben Bitdiddle then implemented the `reverse_array` function to reverse the elements in an array of long integers. Note that `reverse_array` uses his previously implemented `swap` as a helper function. Ben's `reverse_array` C function and its corresponding assembly are shown below:

```
// sz is the number of elements in array a
1 void reverse_array(long *a, long sz)
2 {
3     long s = 0;
4     long e = sz-1;
5     while (s < e) {
6         swap((long *)a[s], (long *)a[e]);
7         s++;
8         e--;
9     }
10 }
```

```
1 reverse_str:
2     pushq    %r12
3     pushq    %rbp
4     pushq    %rbx
5     movq     %rdi, %r12
6     leaq     -1(%rsi), %rbx
7     movq     $0, %rbp
8 .L3:
9     cmpq     %rbx, %rbp
10    jge      .L6
11    movq     (%r12,%rbx), %rsi
12    movq     (%r12,%rbp), %rdi
13    call     swap
14    addq     $1, %rbp
15    subq     $1, %rbx
16    jmp      .L3
17 .L6:
18    popq     %rbx
19    popq     %rbp
20    popq     %r12
21    ret
```

C. (2 points) Where are the C variables `s` and `e` stored at, respectively?

D. (1 point) Which line of assembly computes the condition that determines whether the loop in `reverse_array` should terminate or continue?

E. (1 point) Which line(s) of assembly copy the values to be the arguments used by `swap` function?

F. (2 points) Which C variable does `%r12` contain in the body of the loop?

1. `a`
2. `s`
3. `e`
4. `sz`
5. None of the above

G. (4 points) Suppose `%rsp` is `0x00007fffffff20` *before* executing the first instruction of `reverse_str` (aka line 2 of assembly). What is the value of `%rsp` *immediately before* and *immediately after* executing line 13 of assembly?

Value of `%rsp` immediately before: _____

Value of `%rsp` immediately after: _____

Finally, Ben Bitdiddle writes the following program to test the correctness of `reverse_array`.

```
int
main()
{
    long a[4] = {1, 2, 3, 4};
    reverse_array(a, 4);
    assert(a[0] == 4);
    assert(a[1] == 3);
    assert(a[2] == 2);
    assert(a[3] == 1);
    return 0;
}
```

H. (2 points) Ben found out that running his program results in segmentation fault, and the offending instruction is the first instruction of `swap`, aka `movq (%rdi), %rax`. If Ben is to print out the value of `%rdi` in GDB, what is the value?

I. (2 points) Please fix Ben's bug. You may directly write your corrections on the given C functions.

4 More C Programming (6 points):

Please complete the following code to implement a function `hex2int` that converts a hex string to its integer value (by interpreting the bit pattern represented by the hex string as 2's complement).

```
// return the integer value of the ASCII hex digit c
// you may assume hex digits are always given in lower case
char hex2digit(char c) {

}

int hex2int(char *s) {
    assert(strlen(s) == 8); //assume the string always contains exactly 8 hex "digits"

}

void main() {
    char *s;
    s = "ffffffff";
    int x = hex2int(s);
    assert(x == -1);
    s = "0000000f";
    x = hex2int(s);
    assert(x == 15);
}
```

1. (3 points) Please complete the helper function `hex2digit` that converts a hex character to an integer value in the range $[0, 15]$. You may assume that hex characters are always in lower case.
2. (3 points) Complete `hex2int` to convert a hex string representing an integer in 2's complement to the corresponding integer. The expected return value of this function is demonstrated using two example inputs in `main`. You may assume that the hex string is always exactly 8 characters long and in lower case.

5 Combinatorial Logic (12 points):

A. (2 pts) Prove the equality of boolean expressions, $X \cdot Y + X \cdot \overline{Y} = X$.

B. (5 pts) You are to implement a combinatorial circuit for a 1-bit adder. The circuit takes three one-bit inputs a_i, b_i, c_i^{in} , where a_i, b_i correspond to the bit values at position i of the two binary numbers to be added, and c_i^{in} is the carry-in bit at position i , and produces two output bits s_i, c_{i+1}^{out} where s_i is the bit value of the final sum at position i and c_{i+1}^{out} is the carry-out bit at position i .

1. Draw the truth table of the 1-bit adder circuit.
2. Write the boolean expression to compute s_i in the “sum-of-products” form based on the truth table, without simplification.
3. Write the boolean expression to compute c_{i+1}^{out} in the “sum-of-products” form based on the truth table, without simplification.

4. Prove why the boolean expression in your previous answer (3) is equivalent to $a_i \cdot c_i^{in} + b_i \cdot c_i^{in} + a_i \cdot b_i$

C. (5 points) Use the 1-bit adder circuit in **B.** to build a four-bit adder that adds two 4-bit *unsigned integers* $S = A + B$, where $A = a_3a_2a_1a_0$ and $B = b_3b_2b_1b_0$. The inputs to this four-bit adder are $a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0$ and the outputs are s_3, s_2, s_1, s_0 and *of* (which should be set to 1 if $A + B$ overflowed and 0 otherwise). Use a square box to represent your 1-bit adder and label its input and output ports clearly.

6 FSM (10 points)

You are to implement a FSM circuit that lets a user use a joystick to control 4 lights arranged horizontally. At any given time, exactly one of the 4 lights is lit. We use four one bit variables y_3, y_2, y_1, y_0 to denote the status of the four lights at four positions 3...0 from left to right, with 0 indicating “off” and 1 indicating “on”. We use 1-bit variable x to denote the joystick status with 0 indicating being pushed to the left and 1 indicating being pushed to the right.

If the joystick is pushed to the left, the light immediately to the left of the currently lit light will be lit next. If the currently lit light is already the left light, the lighting configuration remains unchanged. If the joystick is pushed to the right, the light immediately to the right of the currently lit light will be lit next. If the currently lit light is already the right light, the lighting configuration remains unchanged.

We use two bits to denote the FSM state s_1s_0 :

- $s_1s_0 = 00$: the light at position 0 (aka rightmost light) is lit
- $s_1s_0 = 01$: the light at position 1 is lit
- $s_1s_0 = 10$: the light at position 2 is lit
- $s_1s_0 = 11$: the light at position 3 (aka leftmost light) is lit.

A. (3 pts) The FSM starts in its starting state $s_1s_0 = 00$ (aka the rightmost light is lit). Fill in the state *before* processing each input and the corresponding output at each time step.

Joystick input sequence: 0, 1, 1, 1

Step	x	s_1	s_0	y_3	y_2	y_1	y_0
1	0	0	0				
2	1						
3	1						
4	1						

B. (4 pts) Draw the truth table for the FSM.

x	s_1	s_0	$next_s_1$	$next_s_0$	y_3	y_2	y_1	y_0
0	0	0						
0	0	1						
0	1	0						
0	1	1						
1	0	0						
1	0	1						
1	1	0						
1	1	1						

C. (2 pts) Write the boolean expression to compute $next_s_1$ given x, s_1, s_0 based on the truth table as a “sum of products”. Do not simplify the expression.

D. (1 pt) Instead of building the circuit to compute each of y_3, y_2, y_1, y_0 according to its “sum of products” expression derived from the truth table, we notice a relationship between $(next_s_1, next_s_0)$ and (y_3, y_2, y_1, y_0) and use a known circuit to compute y_3, y_2, y_1, y_0 based on $(next_s_1, next_s_0)$ instead. What is this circuit? (Hint: the circuits you’ve learnt include AND, OR, NOT, NAND, XOR, Decoder, Multiplexer and n-bit adder)

7 Non-Pipelined and Pipelined CPU (12 points):

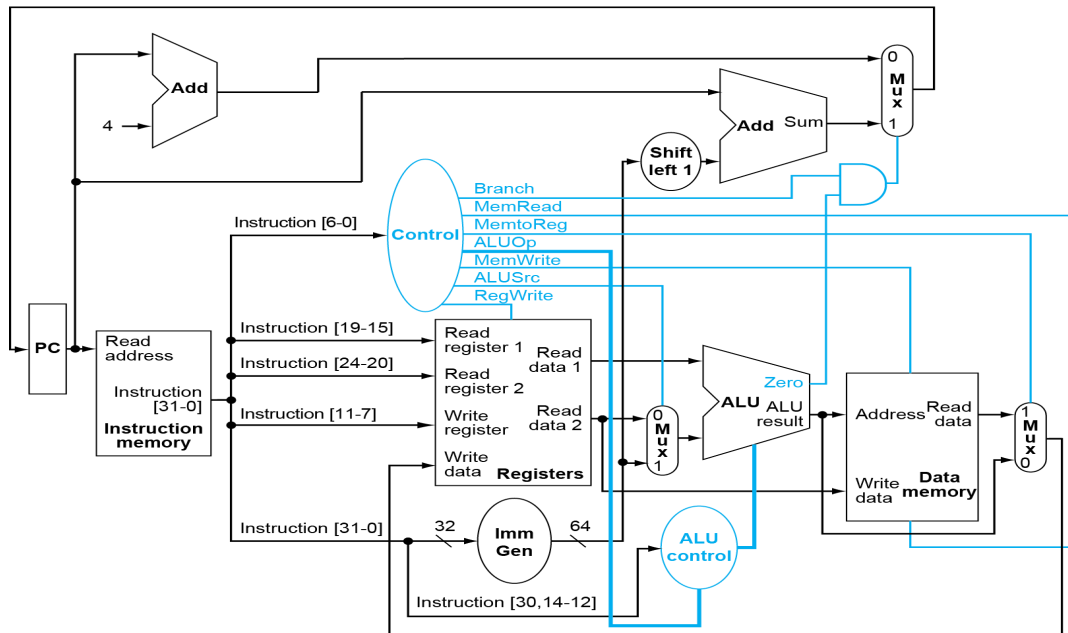


Figure 1: Non-pipelined RISC-V CPU design

A. (4 pts) Suppose the RISC-V instruction being executed is `ld x5, 40(x6)`, which reads 8 bytes from memory to register as $x5 = \text{Memory}[x6 + 40]$. Answer the following questions about the execution of this instruction based on Figure 1:

1. What is the value corresponding to Instruction[11-7] that are fed into the "write register" pins of the RegisterFile?
2. Which input port (0 or 1) of the Mux that connects to the ALU unit is selected according to ALUSrc control signal?
3. What is the actual data value to input port 1 on the Mux that connects to the ALU unit?
4. Which input port (0 or 1) of the bottom right Mux is selected according to MemtoReg control signal?

B. (4 points) The latency of each datapath component is given below. (1 ps = 10^{-12} seconds.)

Component	Latency
① Instruction memory read	200 ps
② Register file read	100 ps
③ ALU	200 ps
④ Data memory read	200 ps
⑤ Register file write	100 ps

1. Suppose one builds a 5-stage pipelined CPU with each component being a separate pipeline stage, what should the clock cycle be? What is each instruction's execution latency? What is the maximum throughput in terms of instructions/sec?
2. Suppose one builds a 3-stage pipelined CPU with the first stage consisting of ①,② and the second stage consisting of ③, and the third stage consisting of ④,⑤, what should the clock cycle be? What is each instruction's execution latency? What is the maximum throughput in terms of instructions/sec?

C. (4 points) The five stages of a pipelined RISC-V CPU are: IF (instruction fetch), ID (instruction decode & register read), EX (execute / compute address), MEM (memory access), WB (write-back).

Consider the following instruction sequence:

- ① `ld x1, 16(x2) //x1=Mem[x2+16]`
- ② `add x3, x1, x4 //x3=x1+x4`
- ③ `sd x5, 8(x2) //Mem[x2+8] = x5`

Assume **no forwarding**. A register may be read and written in the same clock cycle (the read returns the value being written that same cycle).

(i) (1 pt) Identify any data hazard(s) in the sequence. State which pair(s) of instructions are involved and which register causes the hazard.

(ii) (3 pts) Complete the pipeline execution diagram below, inserting `no-op` rows for each stall. You may leave stage cells blank for `no-op` rows.

Instruction	cycle 1	cycle 2	cycle 3	cycle 4	cycle 5	cycle 6	cycle 7	cycle 8	cycle 9	cycle 10	cycle 11

8 Memory Hierarchy (8 points)

Consider a CPU with 32-bit address space. It has a cache with total size 1KB. The cache line (aka cache block) size is 64-byte.

A. (2 points) How many cache lines does the cache contain?

B. (3 pts) Suppose the cache is a direct mapped cache.

1. Split a 32-bit address into fields and label the bits used for the byte offset within a cache line, the cache line index and the tag.
2. For memory address `0xab345f78`, what is its cache line index?
3. How many cacheline-aligned memory addresses can be mapped to the *same* cache location as `0xab345f78`?
4. If more than 1 memory address can be mapped to the same cache line index, how can we determine if the data stored at a given cache line index actually corresponds to the specific memory address being accessed?

C. (3 points) Suppose the cache is a 2-way associative cache, i.e. the cache is organized into sets each of which contains 2 cache lines.

1. How many sets does the cache contain?

2. Split a 32-bit address into fields and label the bits used for the byte offset within a cache line, the cache set index, and the tag.

3. For 32-bit memory address `0xab345f78`, what is its cache set index?

4. How many cacheline-aligned memory addresses can be mapped to the *same* set as `0xab345f78`?

9 Virtual Memory (6 points)

A. (2 pts) Which of the following statements about virtual to physical address translation is true?

1. The translation from virtual to physical address is done by the OS.
2. The translation from virtual to physical address is done by the MMU hardware.
3. The translation from virtual to physical address is done at the granularity of pages.
4. The translation from virtual to physical address is done at the granularity of bytes.
5. The translation from virtual to physical address is done at the granularity of cachelines.

B. (2 pts) Which of the following statements is true?

1. Each running process has its own page table.
2. All running processes share the same page table.
3. 64-bit x86 CPU uses a 4-level page table design.
4. 64-bit x86 CPU uses a flat (aka 1-level) page table design.

C. (2 pts) Suppose the CPU has 12-bit (virtual and physical) addresses with 2-level address translation. The page size is 256 bytes and the page table entry size (PTE) is 2 bytes.

1. Split a 12-bit address into fields and label the bits used for the byte offset within a page, the L1 offset used to index the second level page table and the L0 offset used to index the top level page table.
2. Given a 12-bit address space, what is the maximum memory consumption allowed by a process? What is the total size of page tables required to support such a process?
3. Suppose a process only uses a portion of its address space from $[0 \times 000, 0 \times 01f]$ (aka only addresses in the range of $[0 \times 000, 0 \times 01f]$ are valid) What is the total size of page tables required to support such a process?

—END of Quiz II—

10 Appedix: X86 Cheatsheet

10.1 Registers

x86 registers are 8-bytes. Additionally, the lower order bytes of these registers can be independently accessed as 4-byte, 2-byte, or 1-byte register. The register names are:

8-byte register	Bytes 0-3	Bytes 0-1	Byte 0 (lowest order byte)
%rax	%eax	%ax	%al
%rbx	%ebx	%bx	%bl
%rcx	%ecx	%cx	%cl
%rsi	%esx	%si	%sil
%rdi	%edi	%di	%dil

...the rest is omitted...

10.2 Instructions

Instruction suffixes:

"byte" (b)	1-byte
"word" (w)	2-bytes
"doubleword" (l)	4-bytes
"quadword" (q)	8-bytes

Complete memory addressing mode: A memory operand of the form $D(Rb, Ri, S)$ accesses memory at address $D + \text{val}(Rb) + \text{val}(Ri) * S$, where $\text{val}(Rb)$ and $\text{val}(Ri)$ refer to the value of registers Rb and Ri respectively, D is a constant, and S is a constant of value 1, 2, 4, or 8.

Sign extension and zero extension:

$\text{movzlb } S, D$	copy 1-byte-sized S to 8-byte-sized D and fill in the higher order 7 bytes of D with zero bytes.
$\text{movslq } S, D$	copy 4-byte-sized S to 8-byte-sized D and sign extend the higher order 4 bytes of D , i.e. fill with 0s if S 's sign bit is zero and fill with 1s if S 's sign bit is one.

Basic Arithmetic instructions that you might not remember:

$\text{sal / shl } k, D$	Left shift destination D by k bits
sar	Arithmetic right shift destination D by k bits
shr	Logical right shift destination D by k bits

Jump instructions:

Jump instruction following `cmp S, D`:

<code>jmp</code>	Unconditional jump
<code>je</code>	Jump if D is equal to S
<code>jne</code>	Jump if D is not equal to S
<code>jg</code>	Jump if D is greater than S (signed)
<code>jge</code>	Jump if D is greater or equal than S (signed)
<code>jl</code>	Jump if D is less than S (signed)
<code>jle</code>	Jump if D is less or equal than S (signed)
<code>ja</code>	Jump if D is above S (unsigned)
<code>jae</code>	Jump if D above or equal S(unsigned)
<code>jb</code>	Jump is D is below S (unsigned)
<code>jbe</code>	Jump if D is below or equal S (unsigned)

10.3 Calling convention

Argument Passing:

Which argument	Stored in register
1	<code>%rdi</code>
2	<code>%rsi</code>
3	<code>%rdx</code>
4	<code>%rcx</code>
5	<code>%r8</code>
6	<code>%r9</code>
7 and up	passed on stack

Return value (if any) is stored in `%rax`

Caller save registers: `%rax`, `%rcx`, `%rdx`, `%rdi`, `%rsi`, `%r8-11`

Callee save registers: `%rbx`, `%rbp`, `%r12-15`